

Relatório de Atividade Prática**NÚMERO DO LABORATÓRIO** #Escolher um item.**Integrantes do Grupo (EM ORDEM ALFABÉTICA)**

Nome: Dandi Mateus Aguiar Reis

RA: 10444938

Nome: Leandro Solovjovas dos Santos

RA: 10438426

Nome: Pedro Henrique Marcues Ferreira

RA: 10723681

EXERCÍCIO 1**PRINT DO CÓDIGO FONTE**

```
1 # =====
2 # Laboratório 1 - Exercício 1
3 # Calculadora em Assembly MIPS
4 #
5 # Dandi Mateus Aguiar Reis          RA: 10444938
6 # Leandro Solovjovas dos Santos     RA: 10438426
7 # Pedro Henrique Marques Ferreira   RA: 10723681
8 # =====
9
10 .data
11
12 menu: .asciiz "\n===== MENU =====\n1. Soma\n2. Subtracao\n3. Multiplicacao\n4. Divisao\n5. Sair\nEscolha uma opcao: "
13
14 msg1: .asciiz "\nDigite o primeiro numero: "
15 msg2: .asciiz "Digite o segundo numero: "
16
17 msgRes: .asciiz "\nResultado: "
18
19 msgQuo: .asciiz "\nQuociente: "
20 msgResto: .asciiz "\nResto: "
21
22 erroMenu: .asciiz "\nOpcao invalida!\n"
23 erroDiv: .asciiz "\nErro! Divisao por zero nao permitida.\n"
24
25 novaLinha: .asciiz "\n"
26
```

```
27 .text
28 .globl main
29
30 #####
31 # Programa Principal
32 #####
33
34 main:
35
36 #####
37 # Loop principal do menu
38 #####
39
40 menuLoop:
41
42     # Exibe o menu
43     li $v0, 4
44     la $a0, menu
45     syscall
46
47     # Lê a opção escolhida
48     li $v0, 5
49     syscall
50     move $t0, $v0
51
52     # Seleciona a operação
53     beq $t0, 1, soma
54     beq $t0, 2, subtracao
55     beq $t0, 3, multiplicacao
56     beq $t0, 4, divisao
57     beq $t0, 5, sair
58
```

```

59     # Caso a opção seja inválida
60     li $v0, 4
61     la $a0, erroMenu
62     syscall
63
64     j menuLoop
65
66 #####
67 # Rotina para leitura dos dois operandos
68 #####
69
70 lerNumeros:
71
72     # Primeiro número
73     li $v0, 4
74     la $a0, msg1
75     syscall
76
77     li $v0, 5
78     syscall
79     move $s0, $v0
80
81     # Segundo número
82     li $v0, 4
83     la $a0, msg2
84     syscall
85
86     li $v0, 5
87     syscall
88     move $s1, $v0
89
90     jr $ra

```

```

91 |
92 #####
93 # Soma
94 #####
95
96 soma:
97
98     jal lerNumeros
99
100     add $t1, $s0, $s1
101
102     li $v0, 4
103     la $a0, msgRes
104     syscall
105
106     li $v0, 1
107     move $a0, $t1
108     syscall
109
110     li $v0, 4
111     la $a0, novaLinha
112     syscall
113
114     j menuLoop
115

```

```

116 #####
117 # Subtração
118 #####
119
120 subtracao:
121
122     jal lerNumeros
123
124     sub $t1, $s0, $s1
125
126     li $v0, 4
127     la $a0, msgRes
128     syscall
129
130     li $v0, 1
131     move $a0, $t1
132     syscall
133
134     li $v0, 4
135     la $a0, novaLinha
136     syscall
137
138     j menuLoop
139

```

```

140 #####
141 # Multiplicação
142 #####
143
144 multiplicacao:
145
146     jal lerNumeros
147
148     mult $s0, $s1
149     mflo $t1
150
151     li $v0, 4
152     la $a0, msgRes
153     syscall
154
155     li $v0, 1
156     move $a0, $t1
157     syscall
158
159     li $v0, 4
160     la $a0, novaLinha
161     syscall
162
163     j menuLoop
164

```

```

165 #####
166 # Divisão
167 #####
168
169 divisao:
170
171     jal lerNumeros
172
173     # Verifica divisão por zero
174     beq $s1, $zero, erroDivisao
175
176     div $s0, $s1
177
178     mflo $t2      # Quociente
179     mfhi $t3      # Resto
180
181     # Imprime o quociente
182     li $v0, 4
183     la $a0, msgQuo
184     syscall
185
186     li $v0, 1
187     move $a0, $t2
188     syscall
189
190     # Imprime o resto
191     li $v0, 4
192     la $a0, msgResto
193     syscall
194
195     li $v0, 1
196     move $a0, $t3

```

```

195     li $v0, 1
196     move $a0, $t3
197     syscall
198
199     li $v0, 4
200     la $a0, novaLinha
201     syscall
202
203     j menuLoop
204
205 #####
206 # Tratamento de erro: divisão por zero
207 #####
208
209 erroDivisao:
210
211     li $v0, 4
212     la $a0, erroDiv
213     syscall
214
215     j menuLoop
216
217 #####
218 # Encerramento do programa
219 #####
220
221 sair:
222
223     li $v0, 10
224     syscall

```

PRINT COM TESTES DE FUNCIONALIDADE

```
===== MENU =====
1. Soma
2. Subtracao
3. Multiplicacao
4. Divisao
5. Sair
Escolha uma opcao: -1

Opcao invalida!

===== MENU =====
1. Soma
2. Subtracao
3. Multiplicacao
4. Divisao
5. Sair
Escolha uma opcao: 1

Digite o primeiro numero: -1
Digite o segundo numero: 2

Resultado: 1

===== MENU =====
1. Soma
2. Subtracao
3. Multiplicacao
4. Divisao
5. Sair
Escolha uma opcao: 2

Digite o primeiro numero: -2
Digite o segundo numero: 9

Resultado: -11

===== MENU =====
1. Soma
2. Subtracao
3. Multiplicacao
4. Divisao
5. Sair
Escolha uma opcao: 3

Digite o primeiro numero: -9
Digite o segundo numero: 10

Resultado: -90

===== MENU =====
1. Soma
2. Subtracao
3. Multiplicacao
4. Divisao
5. Sair
Escolha uma opcao: 4

Digite o primeiro numero: 10
Digite o segundo numero: 0

Erro! Divisao por zero nao permitida.

===== MENU =====
1. Soma
2. Subtracao
3. Multiplicacao
4. Divisao
5. Sair
Escolha uma opcao: 4

Digite o primeiro numero: 10
Digite o segundo numero: 10

Quociente: 1
Resto: 0

===== MENU =====
1. Soma
2. Subtracao
3. Multiplicacao
4. Divisao
5. Sair
Escolha uma opcao: 5

-- program is finished running --
```

COMENTÁRIOS GERAIS

A solução foi desenvolvida utilizando um menu interativo que permite ao usuário escolher entre as operações de soma, subtração, multiplicação, divisão ou encerrar o programa. Após a execução de cada operação, o programa retorna ao menu principal, permitindo novas operações.

A principal dificuldade foi organizar o fluxo do programa utilizando desvios (beq e j) e implementar corretamente a divisão, exibindo tanto o quociente quanto o resto, além de tratar a divisão por zero.

A solução atende aos requisitos propostos no exercício. O programa realiza as quatro operações básicas, possui tratamento para opção de menu inválida e para divisão por zero, além de permanecer em execução até que o usuário escolha a opção de sair.

Como comentário geral, o código foi organizado em rotinas para facilitar a leitura e manutenção, utilizando comentários para identificar cada etapa da implementação.

EXERCÍCIO 2

PRINT DO CÓDIGO FONTE

```
1  # =====
2  # Laboratório 1 - Exercício 2
3  # Comparação de Três Números
4  #
5  # Dandi Mateus Aguiar Reis           RA: 10444938
6  # Leandro Solovjovas dos Santos      RA: 10438426
7  # Pedro Henrique Marques Ferreira    RA: 10723681
8  # =====
9
10 .data
11
12 msg1: .asciiz "Digite o primeiro numero: "
13 msg2: .asciiz "Digite o segundo numero: "
14 msg3: .asciiz "Digite o terceiro numero: "
15
16 msgMaior: .asciiz "\nMaior valor: "
17 msgMenor: .asciiz "\nMenor valor: "
18 msgMeio: .asciiz "\nValor intermediario: "
19
20 novaLinha: .asciiz "\n"
21
```

```
22 .text
23 .globl main
24
25 #####
26 # Leitura dos três números
27 #####
28
29 main:
30
31     # Primeiro número
32     li $v0,4
33     la $a0,msg1
34     syscall
35
36     li $v0,5
37     syscall
38     move $s0,$v0
39
40     # Segundo número
41     li $v0,4
42     la $a0,msg2
43     syscall
44
45     li $v0,5
46     syscall
47     move $s1,$v0
48
```

```
49     # Terceiro número
50     li $v0,4
51     la $a0,msg3
52     syscall
53
54     li $v0,5
55     syscall
56     move $s2,$v0
57
58 #####
59 # Inicialmente considera o primeiro número como
60 # maior e menor provisórios.
61 #####
62
63     move $t0,$s0    # maior
64     move $t1,$s0    # menor
65
```

```
66 #####
67 # Comparação do segundo número
68 #####
69
70     bgt $s1,$t0,novoMaior1
71     blt $s1,$t1,novoMenor1
72     j comparaTerceiro
73
74 novoMaior1:
75     move $t0,$s1
76     j comparaTerceiro
77
78 novoMenor1:
79     move $t1,$s1
80
```

```

81 #####
82 # Comparação do terceiro número
83 #####
84
85 comparaTerceiro:
86
87     bgt $s2,$t0,novoMaior2
88     blt $s2,$t1,novoMenor2
89     j  achaIntermediario
90
91 novoMaior2:
92     move $t0,$s2
93     j  achaIntermediario
94
95 novoMenor2:
96     move $t1,$s2
97

```

```

98 #####
99 # O intermediário é o valor que não é nem maior
100 # nem menor.
101 #####
102
103 achaIntermediario:
104
105     beq $s0,$t0,testaS1
106     beq $s0,$t1,testaS1
107     move $t2,$s0
108     j  imprime
109
110 testaS1:
111
112     beq $s1,$t0,usaS2
113     beq $s1,$t1,usaS2
114     move $t2,$s1
115     j  imprime
116
117 usaS2:
118
119     move $t2,$s2
120

```

```

121 #####
122 # Impressão dos resultados
123 #####
124
125 imprime:
126
127     # Maior
128     li $v0,4
129     la $a0,msgMaior
130     syscall
131
132     li $v0,1
133     move $a0,$t0
134     syscall
135
136     # Menor
137     li $v0,4
138     la $a0,msgMenor
139     syscall
140
141     li $v0,1
142     move $a0,$t1
143     syscall
144
145     # Intermediário
146     li $v0,4
147     la $a0,msgMeio

```

```

145     # Intermediário
146     li $v0,4
147     la $a0,msgMeio
148     syscall
149
150     li $v0,1
151     move $a0,$t2
152     syscall
153
154     li $v0,4
155     la $a0,novaLinha
156     syscall
157
158 #####
159 # Encerramento
160 #####
161
162     li $v0,10
163     syscall

```

PRINT COM TESTES DE FUNCIONALIDADE

```
Digite o primeiro numero: 10
Digite o segundo numero: 20
Digite o terceiro numero: 30

Maior valor: 30
Menor valor: 10
Valor intermediario: 20

-- program is finished running --

Digite o primeiro numero: 30
Digite o segundo numero: 20
Digite o terceiro numero: 10

Maior valor: 30
Menor valor: 10
Valor intermediario: 20

-- program is finished running --

Digite o primeiro numero: -15
Digite o segundo numero: 20
Digite o terceiro numero: 2

Maior valor: 20
Menor valor: -15
Valor intermediario: 2

-- program is finished running --

Digite o primeiro numero: 5
Digite o segundo numero: 5
Digite o terceiro numero: 10

Maior valor: 10
Menor valor: 5
Valor intermediario: 10

-- program is finished running --
```

COMENTÁRIOS GERAIS

Neste exercício, a solução foi desenvolvida para ler três números inteiros distintos e identificar o maior, o menor e o valor intermediário utilizando comparações e desvios condicionais da linguagem Assembly MIPS.

A principal dificuldade foi elaborar a lógica de comparação para que o programa funcionasse corretamente independentemente da ordem em que os números fossem digitados. Também foi necessário identificar corretamente o valor intermediário após determinar o maior e o menor.

A solução atende aos requisitos propostos no exercício. O programa realiza a leitura dos três valores, identifica corretamente o maior, o menor e o intermediário, exibe os resultados com mensagens claras e utiliza comentários para facilitar o entendimento do código.

Como comentário geral, foram realizados testes com diferentes ordens de entrada e com números negativos para verificar o correto funcionamento da lógica implementada.

EXERCÍCIO 3

CÓDIGO FONTE

```
1  # =====
2  # Laboratório 1 - Exercício 3
3  # Análise de Bits e Paridade
4  #
5  # Dandi Mateus Aguiar Reis          RA: 10444938
6  # Leandro Solovjovas dos Santos    RA: 10438426
7  # Pedro Henrique Marques Ferreira  RA: 10723681
8  # =====
9
10 .data
11
12 msgEntrada: .ascii "Digite um numero inteiro: "
13
14 msgPar: .ascii "\nO numero e PAR."
15 msgImpar: .ascii "\nO numero e IMPAR."
16
17 msgBitOn: .ascii "\nO 5º bit esta ON."
18 msgBitOff: .ascii "\nO 5º bit esta OFF."
19
20 novaLinha: .ascii "\n"
21
22 .text
23 .globl main
24
25 #####
26 # Programa Principal
27 #####
28
29 main:
30
31     # Solicita um número ao usuário
32     li $v0,4
33     la $a0,msgEntrada
34     syscall
35
36     # Lê o número
37     li $v0,5
38     syscall
39     move $s0,$v0
40
41 #####
42 # Verificação de Paridade
43 # Utiliza máscara 1 (00000001)
44 #####
45
46     andi $t0,$s0,1
47
48     beq $t0,$zero,numeroPar
49
50 #####
51 # Número Ímpar
52 #####
53
54 numeroImpar:
55
56     li $v0,4
57     la $a0,msgImpar
58     syscall
59
60     j verificaBit5
61
62 #####
63 # Número Par
64 #####
65
66 numeroPar:
67
68     li $v0,4
69     la $a0,msgPar
70     syscall
71
72 #####
73 # Verificação do 5º bit
74 # Bit 5 -> máscara 32 (00100000)
75 #####
76
77 verificaBit5:
78
79     andi $t1,$s0,32
80
81     beq $t1,$zero,bitOff
82
```

```

83 #####
84 # Bit ligado
85 #####
86
87 bitOn:
88
89     li $v0,4
90     la $a0,msgBitOn
91     syscall
92
93     j fim
94
95 #####
96 # Bit desligado
97 #####
98
99 bitOff:
100
101     li $v0,4
102     la $a0,msgBitOff
103     syscall
104
105 #####
106 # Encerramento
107 #####
108
109 fim:
110
111     li $v0,4
112     la $a0,novaLinha
113     syscall
114
115     li $v0,10
116     syscall

```

TESTES DE FUNCIONALIDADE

Digite um numero inteiro: -3

O numero e IMPAR.
O 5° bit esta ON.

-- program is finished running --

Digite um numero inteiro: -10

O numero e PAR.
O 5° bit esta ON.

-- program is finished running --

Digite um numero inteiro: 10

O numero e PAR.
O 5° bit esta OFF.

-- program is finished running --

Digite um numero inteiro: 33

O numero e IMPAR.
O 5° bit esta ON.

-- program is finished running --

COMENTÁRIOS GERAIS

Neste exercício, a solução foi desenvolvida para analisar um número inteiro utilizando operações lógicas da linguagem Assembly MIPS. O programa verifica se o número é par ou ímpar por meio da operação AND com a máscara 1 e identifica se o 5º bit está ligado (ON) ou desligado (OFF) utilizando a máscara 32.

A principal dificuldade foi compreender a utilização das máscaras de bits e aplicar corretamente as operações lógicas para verificar a paridade e o estado do 5º bit.

A solução atende aos requisitos propostos no exercício. O programa realiza a leitura do número, identifica corretamente se ele é par ou ímpar, verifica o estado do 5º bit e exibe mensagens claras com os resultados.

Como comentário geral, foram realizados testes com números pares, ímpares, positivos e negativos para confirmar o correto funcionamento das operações lógicas implementadas.

OBSERVAÇÕES FINAIS

- **Gerar um PDF** deste arquivo antes de submeter na tarefa do Moodle;
- Além deste relatório, **enviar o arquivo .asm** do(s) programa(s) criado(s): Ex1.asm, Ex2.asm ...
- Cuidado com o tamanho e qualidade das imagens. Observe se é possível visualizar seu conteúdo!
- **ATENÇÃO:** Na tarefa do Moodle, apenas 1 (um) aluno da equipe deve submeter a atividade.
- **O que será avaliado?**
 - Completude, clareza e organização do relatório. É obrigatório usar este *template*;
 - Funcionamento correto dos programas, atendendo ao que foi solicitado. O quão fiel cada programa é quanto à descrição do enunciado;
 - Completude dos testes realizados;
 - Indentação, comentários, organização e legibilidade dos códigos;